

Embedded Programming

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

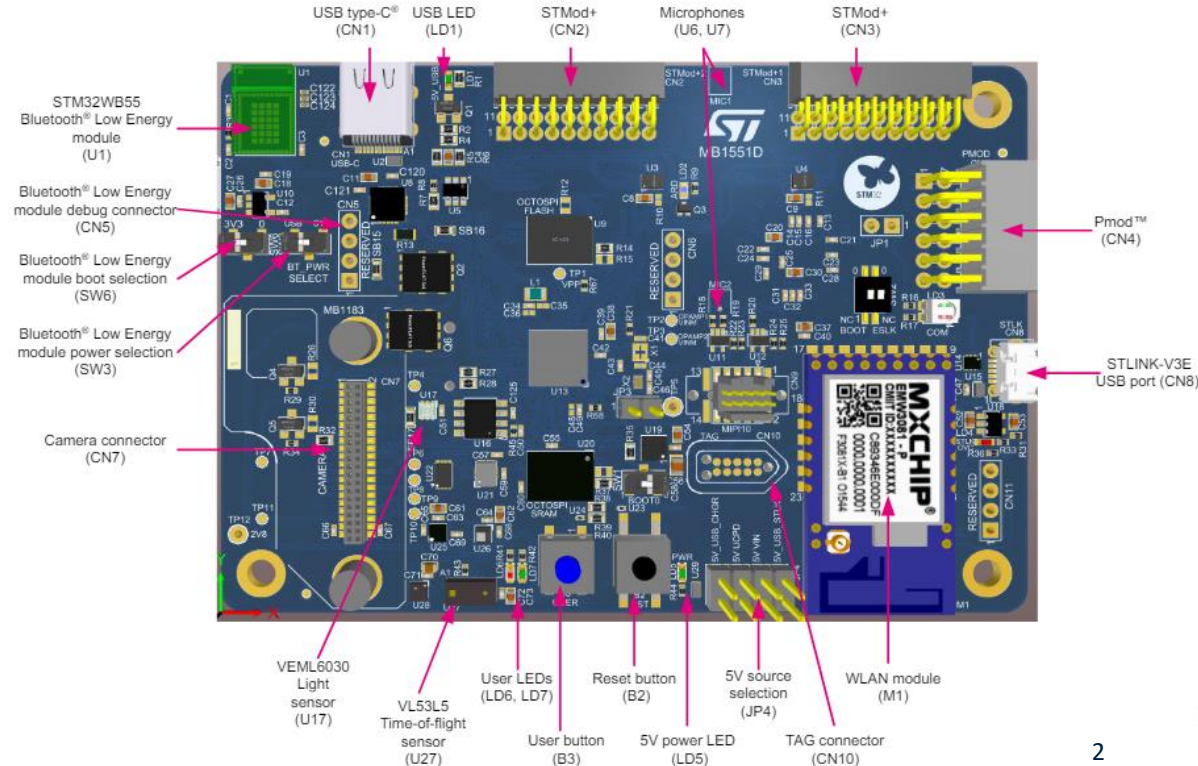
2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl

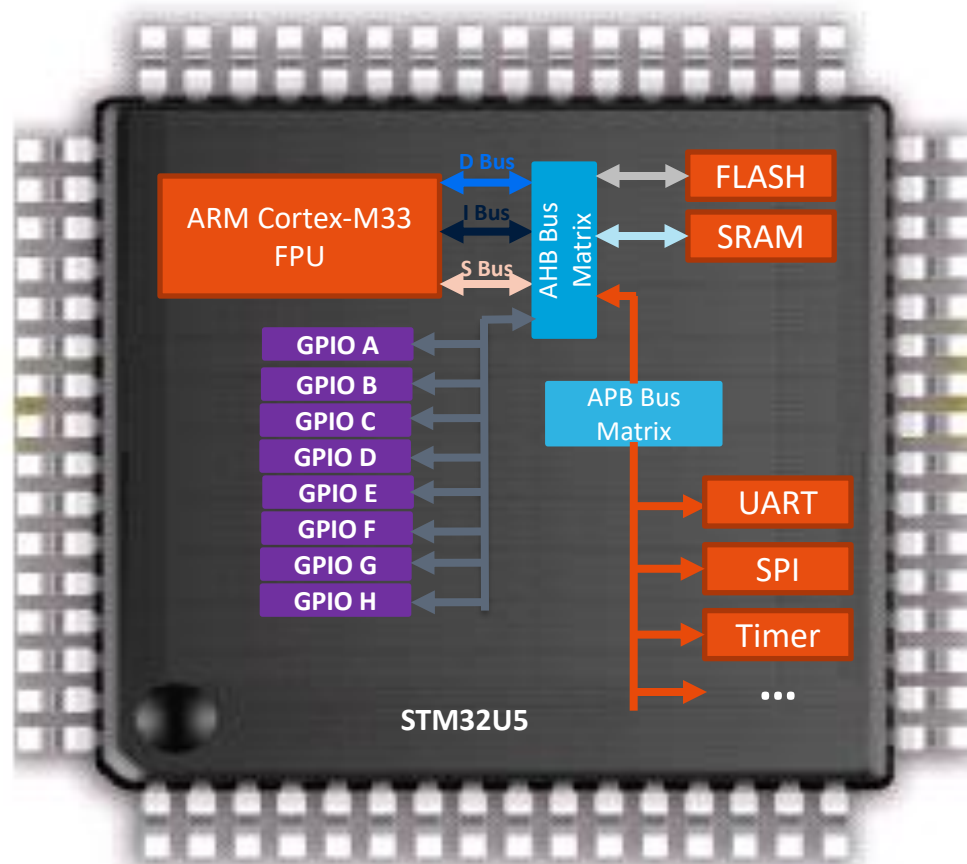


Maastricht University

The STM32U5 architecture



Simplified block diagram



STM32U5

Parallel interface

FSMC 8-/16-bit
(TFT-LCD, SRAM,
NOR, NAND)

Timers

19 timers including:
2 x 16-bit advanced
motor control timers
4 x ULP timers
5 x 16-bit timers
4 x 32-bit timers

I/Os

Touch-sensing controller
Camera Interface

Arm® Cortex®-M33 CPU
160 MHz
TrustZone®
FPU
MPU
ETM

LPDMA

ART Accelerator™

CORDIC

FMAC

Chrom-ART
Accelerator™

Up to 2-Mbyte
Flash memory
Dual Bank

786-Kbyte RAM

Connectivity

USB2.0 OTG_FS, UCPD,
2 x SD/SDIO/MMC, 3 x SPI,
4 x I²C, 1 x CAN FD,
2 x Octo-SPI,
5 x USART + 1 x LPUART

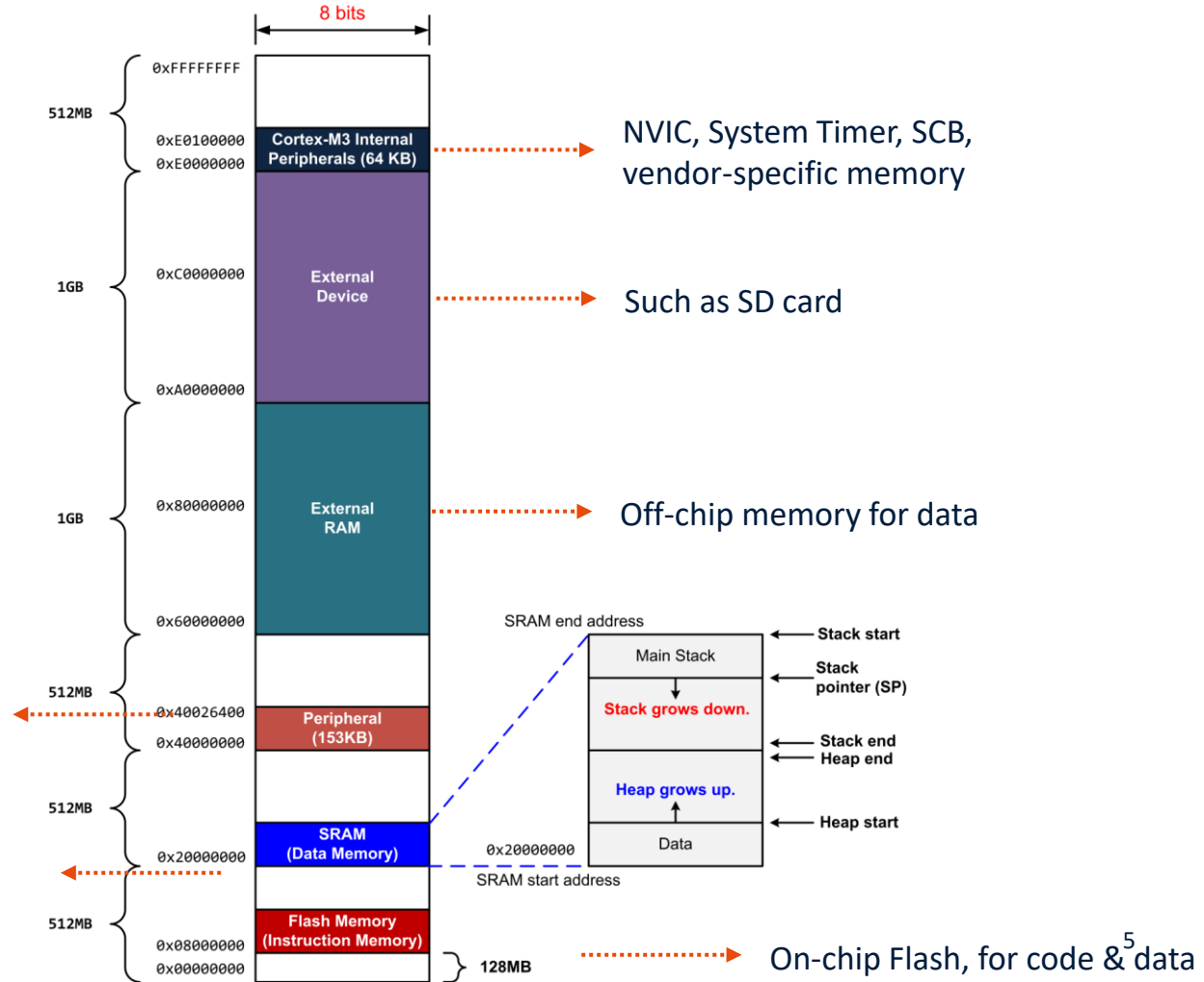
Digital

AES (256-bit),
SHA-1, SHA-256, TRNG,
PKA, HUK, 2 x SAI,
MDF, ADF

Analog

1 x 14-bit ADC 2 MSPS,
1 x 12-bit ADC 2 MSPS,
2 x DAC, 2 x comparators,
2 x op amps
1 x temperature sensor

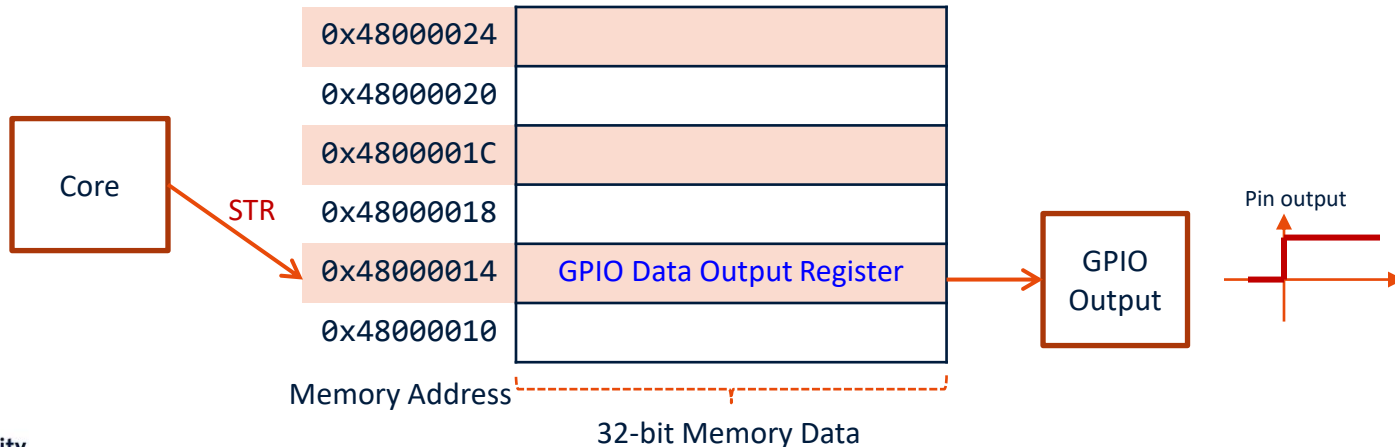
Memory Map



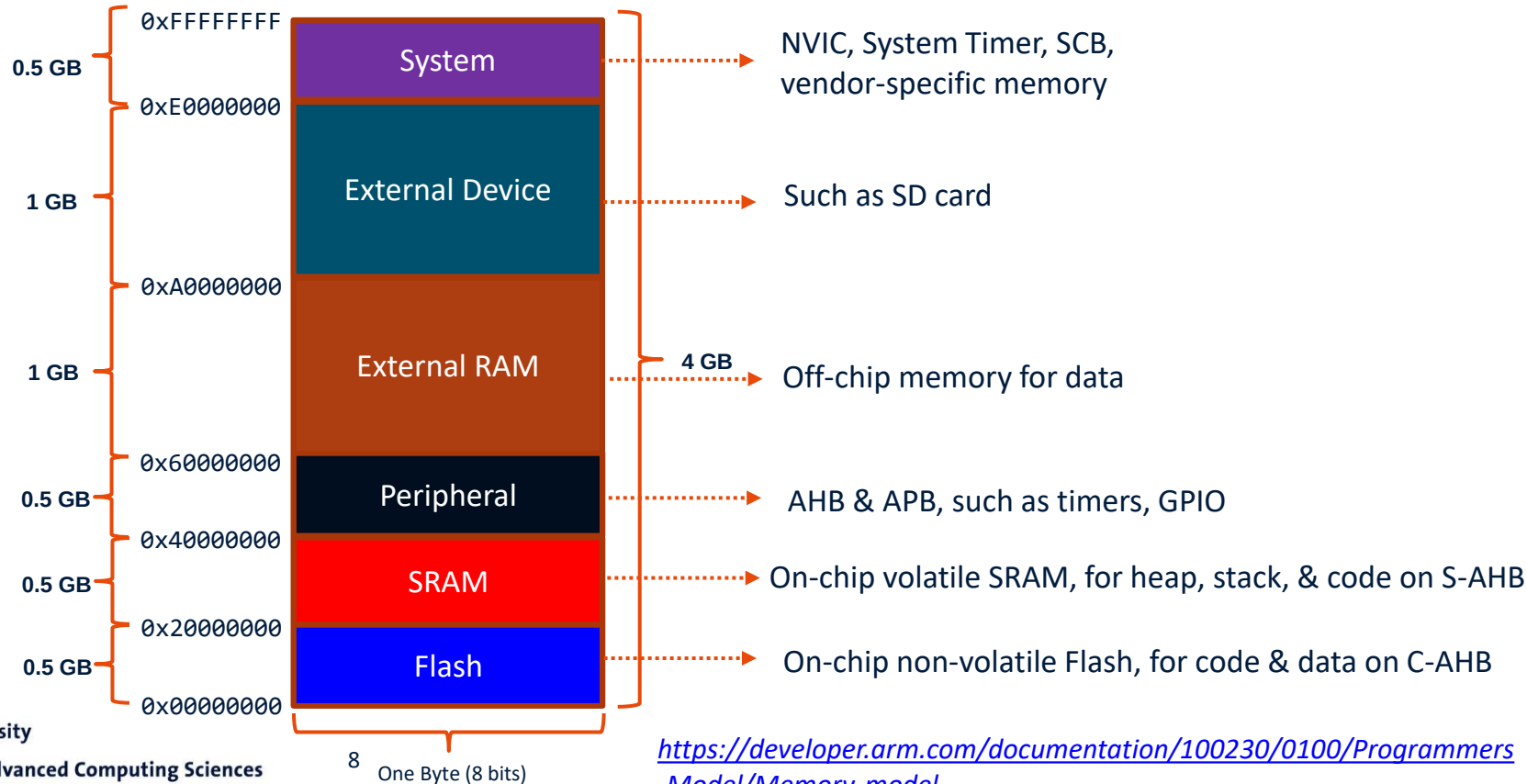
Memory-mapped I/O

Interfacing Peripherals

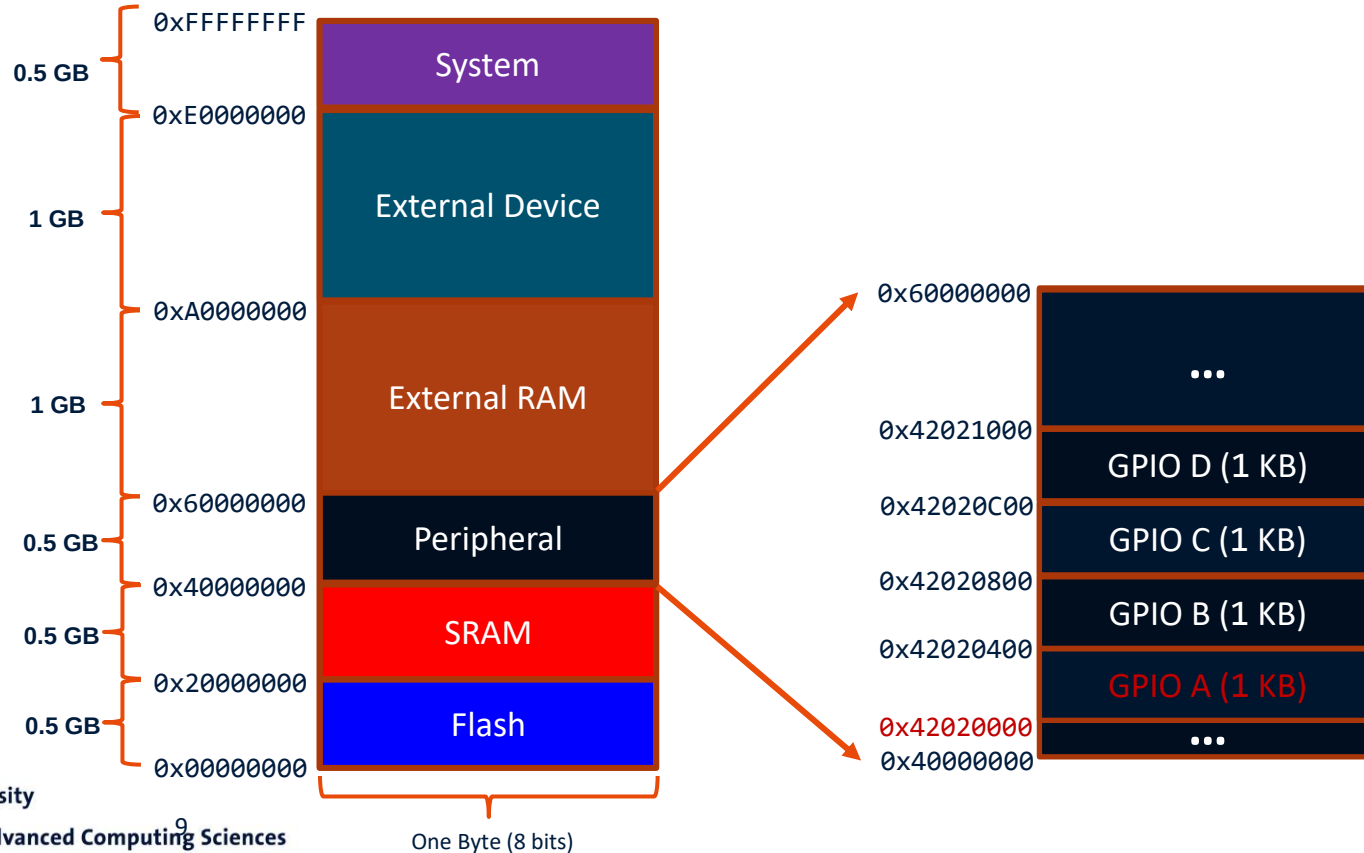
- **Port-mapped I/O**
 - Use special CPU instructions: `Special_instruction Reg, Port`
- **Memory-mapped I/O**
 - A simpler and more convenient way to interface I/O devices
 - Each device registers is assigned to a memory address in the address space of the microprocessor
 - Use native CPU load/store instructions: `LDR/STR Reg, [Reg, #imm]`



Memory Map of Cortex-M33



Memory Map of Cortex-M33



Memory Map of Cortex-M33

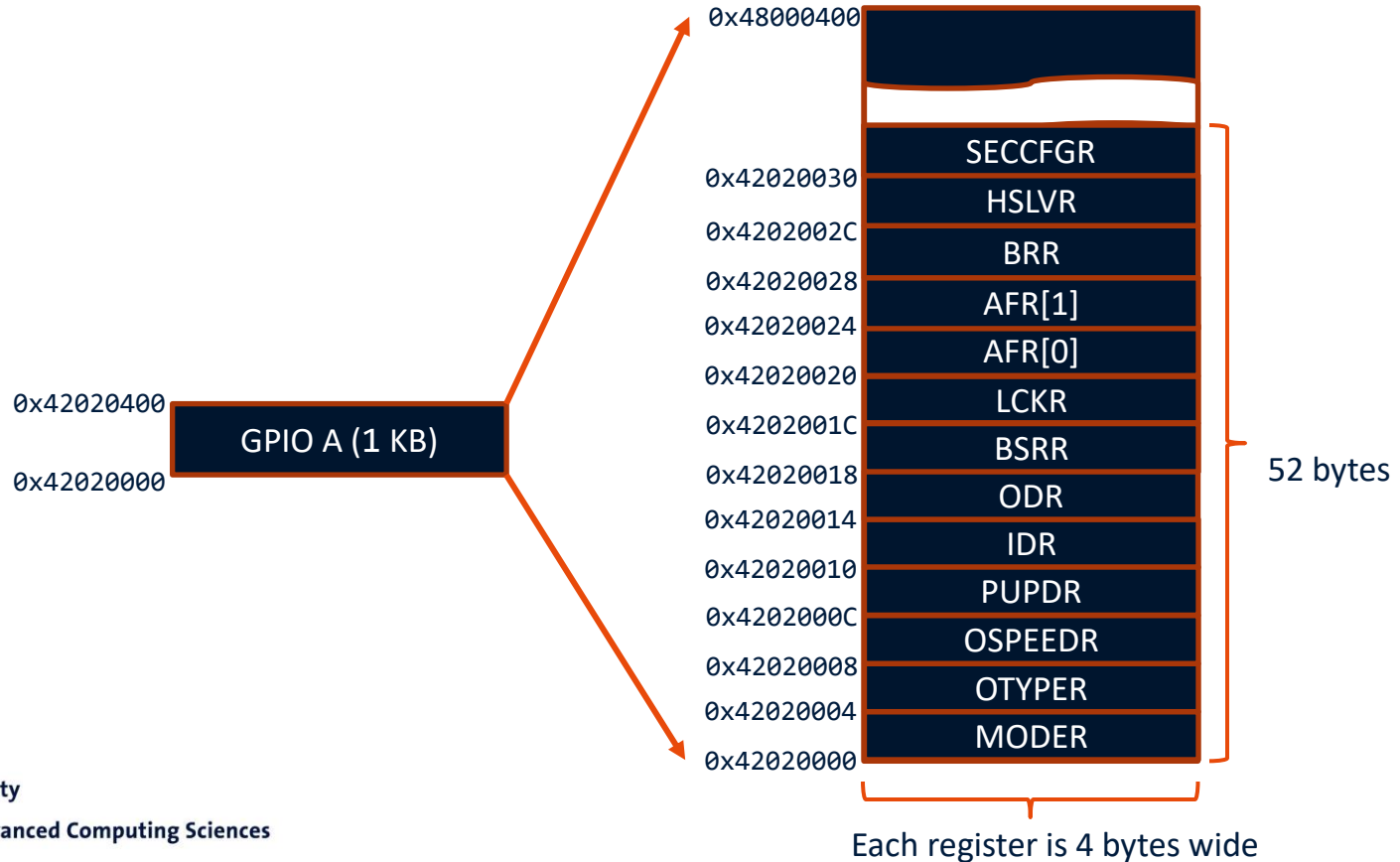


```
stm32u585xx.h
#define PERIPH_BASE_NS      (0x40000000UL) //< Peripheral non-secure base address

#define AHB2PERIPH_BASE_NS  (PERIPH_BASE_NS + 0x02020000UL)

#define GPIOA_BASE_NS       (AHB2PERIPH_BASE_NS + 0x000000UL)
#define GPIOB_BASE_NS       (AHB2PERIPH_BASE_NS + 0x004000UL)
//1KB size (0x400=0d1024)
```

GPIO Memory Map



GPIO Memory Map

stm32u585xx.h

typedef struct

```
{  
  __IO uint32_t MODER; /*!< GPIO port mode register, Address offset: 0x00 */  
  __IO uint32_t OTYPER; /*!< GPIO port output type register, Address offset: 0x04 */  
  __IO uint32_t OSPEEDR; /*!< GPIO port output speed register, Address offset: 0x08 */  
  __IO uint32_t PUPDR; /*!< GPIO port pull-up/pull-down register, Address offset: 0x0C */  
  __IO uint32_t IDR; /*!< GPIO port input data register, Address offset: 0x10 */  
  __IO uint32_t ODR; /*!< GPIO port output data register, Address offset: 0x14 */  
  __IO uint32_t BSRR; /*!< GPIO port bit set/reset register, Address offset: 0x18 */  
  __IO uint32_t LCKR; /*!< GPIO port configuration lock register, Address offset: 0x1C */  
  __IO uint32_t AFR[2]; /*!< GPIO alternate function registers, Address offset: 0x20-0x24 */  
  __IO uint32_t BRR; /*!< GPIO Bit Reset register, Address offset: 0x28 */  
  __IO uint32_t HSLVR; /*!< GPIO high-speed low voltage register, Address offset: 0x2C */  
  __IO uint32_t SECCFGR; /*!< GPIO secure configuration register, Address offset: 0x30 */  
} GPIO_TypeDef;
```

GPIO Memory Map

Suppose we want to set the output of pin 8 of Port A to high. To achieve this, we need to set bit 8 of the output data register (ODR) of GPIOA to 1.

13.4.6 GPIO port output data register (GPIOx_ODR) (x = A to J)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODy**: Port output data I/O pin y (y = 15 to 0)

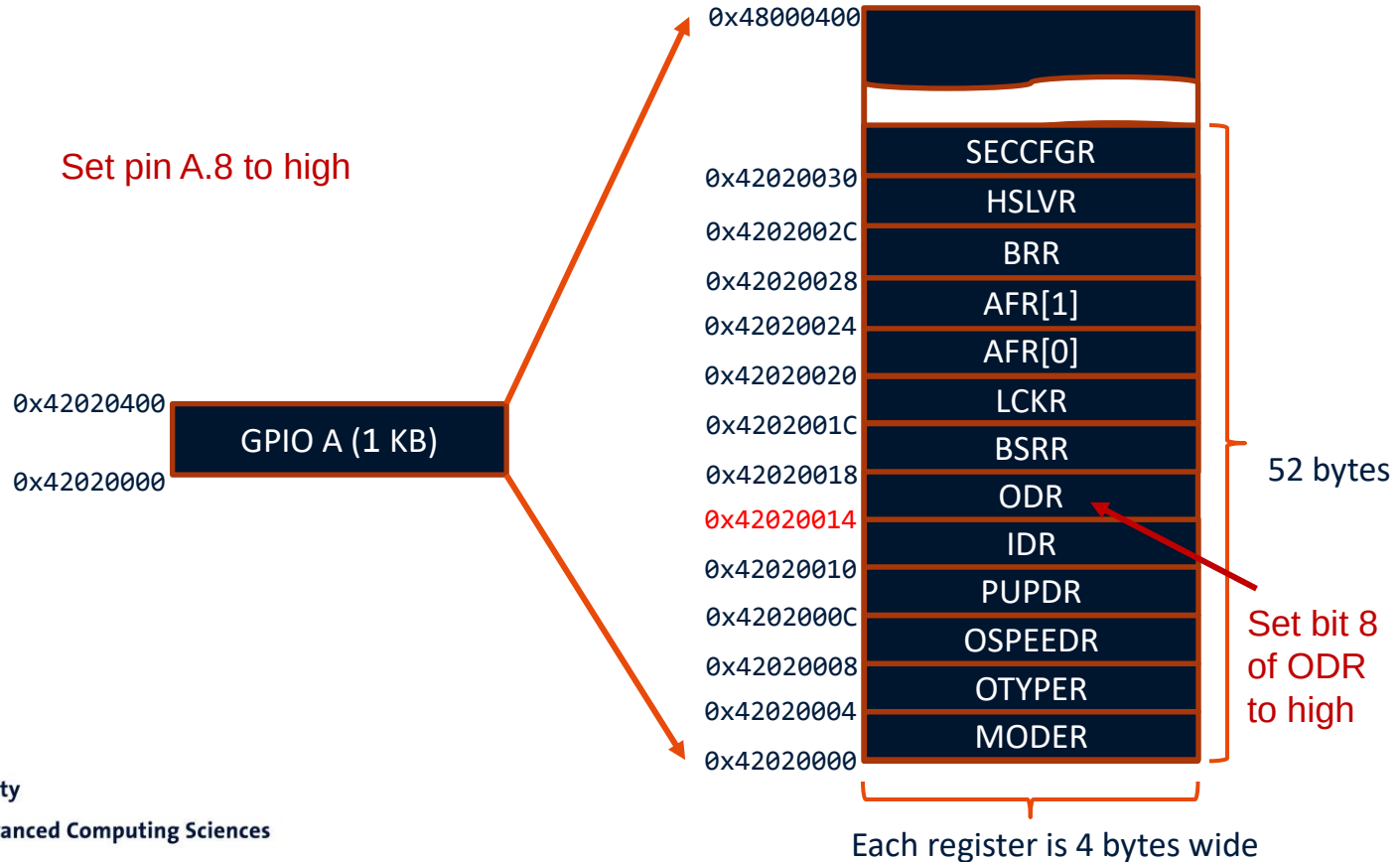
These bits can be read and written by software.

Note: For atomic bit set/reset, these bits can be individually set and/or reset by writing to GPIOx_BSRR or GPIOx_BRR (x = A to J).

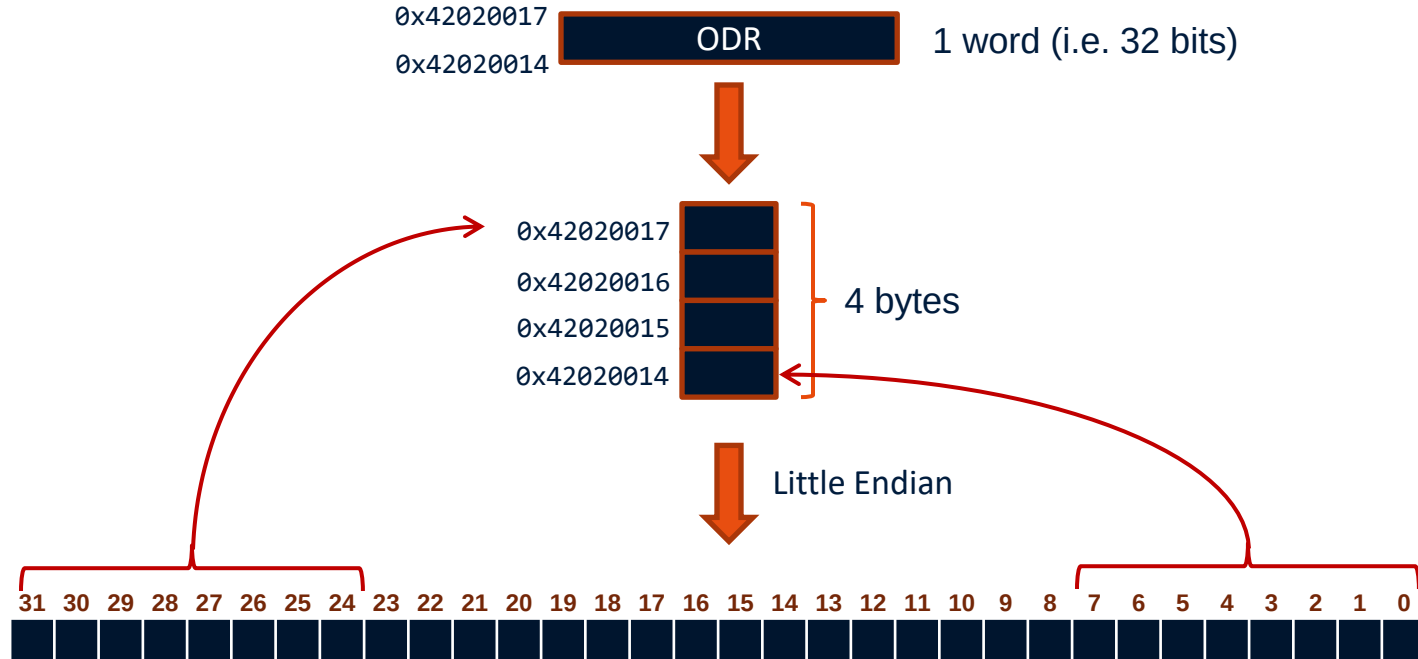
This bit is reserved and must be kept at reset value when the corresponding I/O is not available on the selected package.

RM0456

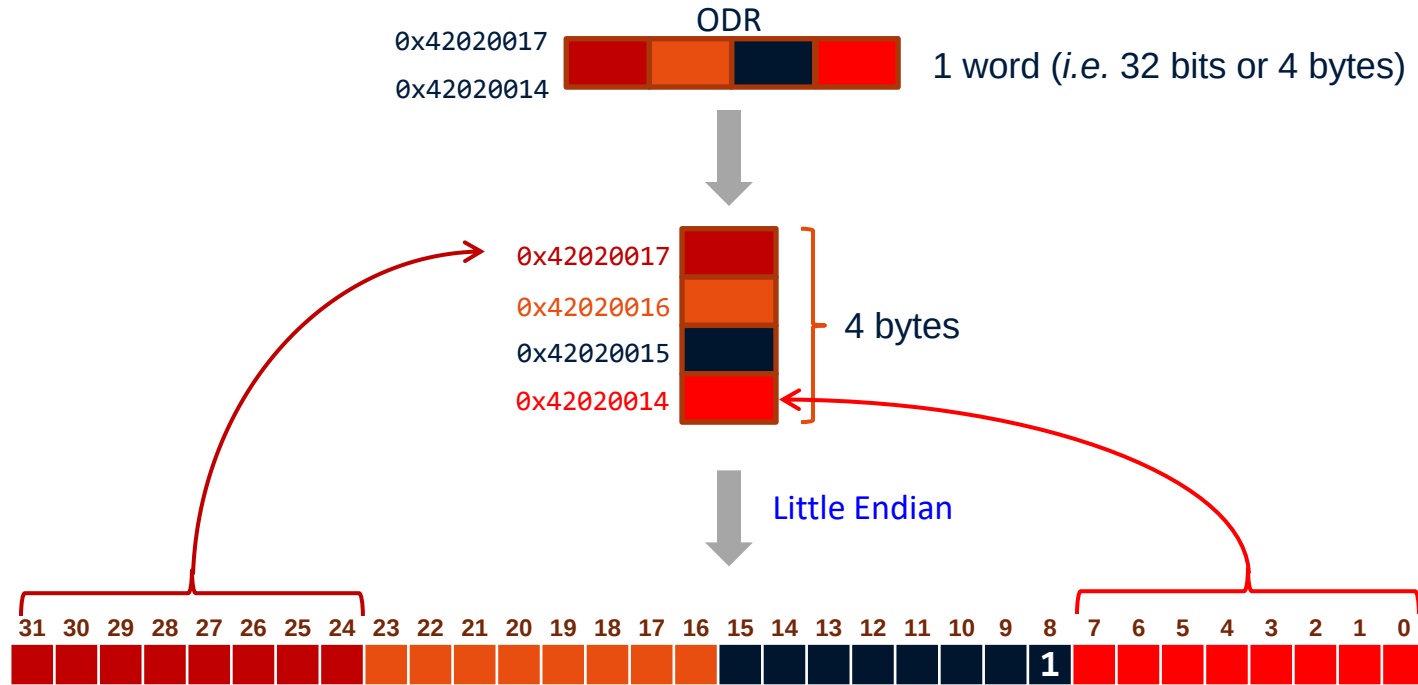
GPIO Memory Map



Output Data Register (ODR)



Output Data Register (ODR)



```
*((uint32_t *) 0x42020014) |= 1UL << 8;
```

Dereferencing a pointer Bitwise OR

Dereferencing a Memory Address

	SECCFGR
0x42020030	HSLVR
0x4202002C	BRR
0x42020028	AFR[1]
0x42020024	AFR[0]
0x42020020	LCKR
0x4202001C	BSRR
0x42020018	ODR
0x42020014	IDR
0x42020010	PUPDR
0x4202000C	OSPEEDR
0x42020008	OTYPER
0x42020004	MODER
0x42020000	

```
typedef struct {  
    volatile uint32_t MODER;    // Mode register  
    volatile uint32_t OTYPER;   // Output type register  
    volatile uint32_t OSPEEDR;  // Output speed register  
    volatile uint32_t PUPDR;    // Pull-up/pull-down register  
    volatile uint32_t IDR;      // Input data register  
    volatile uint32_t ODR;      // Output data register  
    volatile uint32_t BSRR;     // Bit set/reset register  
    volatile uint32_t LCKR;     // Configuration lock register  
    volatile uint32_t AFR[2];   // Alternate function registers  
    volatile uint32_t BRR;      // Bit Reset register  
    volatile uint32_t HSLVR;    // High-speed low voltage  
    register  
    volatile uint32_t SECCFGR;   // Secure configuration  
    register  
} GPIO_TypeDef;  
  
// Casting memory address to a pointer  
#define GPIOA ((GPIO_TypeDef *) 0x42020000)
```

GPIOA->ODR |= 1UL<<8;

or **(*GPIOA).ODR |= 1UL<<8;**

Programming in Assembly

In C

```
GPIOA->ODR |= 1UL<<8; // Set bit 8 to 1
```

*The **EQU** directive gives a symbolic name to a numeric constant*

In Assembly

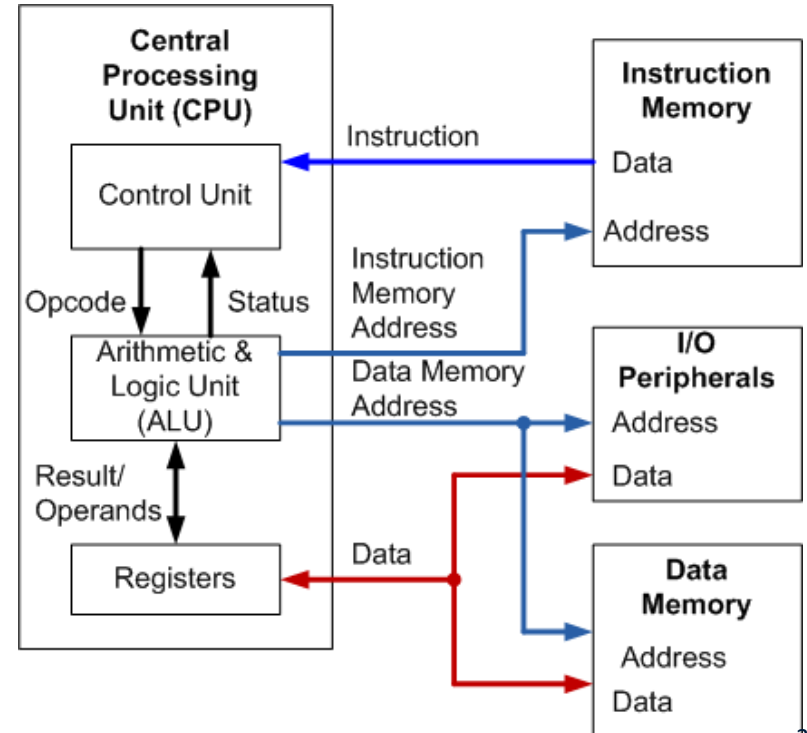
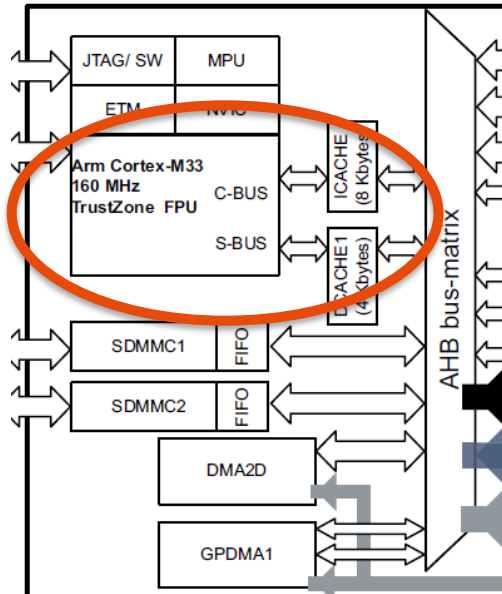


```
.equ GPIOA_ODR, 0x42020000 // GPIOA Output Data Register address
LDR    r0, =GPIOA_ODR      // Load GPIOA->ODR address into r0
LDR    r1, [r0]             // Read current value of GPIOA->ODR
ORR    r1, r1, #0x100       // Set bit 8 (simplified)
STR    r1, [r0]             // Write back modified value
```

The STM32U5 architecture

- Lets have a look at its block diagram: [STM32U585xx block diagram.pdf](https://www.st.com/resource/en/datasheet/stm32u585ai.pdf) (<https://www.st.com/resource/en/datasheet/stm32u585ai.pdf>)
- Main parts:
 - **Core**
 - **Caches**
 - **AHB (High-performance bus)** bus matrix - provides access from a master to a worker, enabling concurrent access and efficient operation even when several high-speed peripherals work simultaneously
 - **APB (Advanced peripheral bus)**
 - **Flash – SRAM**
 - **Timers**
 - **GPIO**
 - **UART/I2C/SPI**

The core



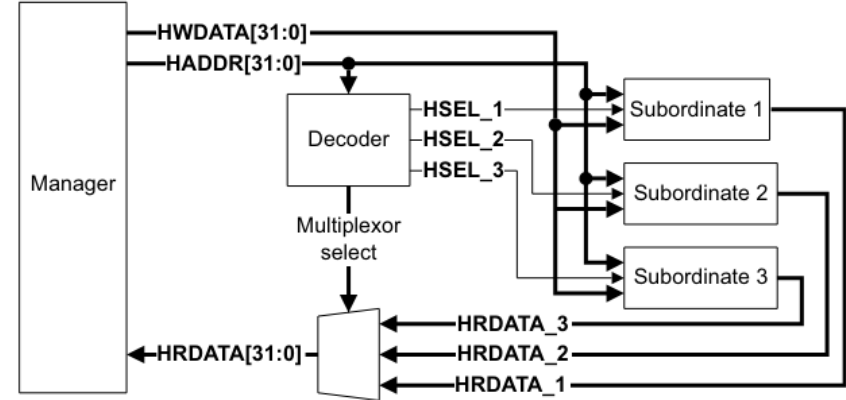
ICACHE and DCACHE

- The ICACHE is introduced on the C-AHB code bus to improve performance when fetching instruction (or data) from both internal and external memories
 - 0 wait-state on **cache hit**
 - Hit-under-miss capability, allowing to serve new processor requests while a line refill (due to a previous cache miss) is still ongoing
 - Hit ratio improved by two-ways set-associative architecture and pLRU-t replacement policy (pseudo-least-recently-used, based on binary tree), algorithm with best complexity/performance balance
- The DCACHE is introduced on the S-AHB system bus to improve the performance of data traffic to/from external memories

System buses

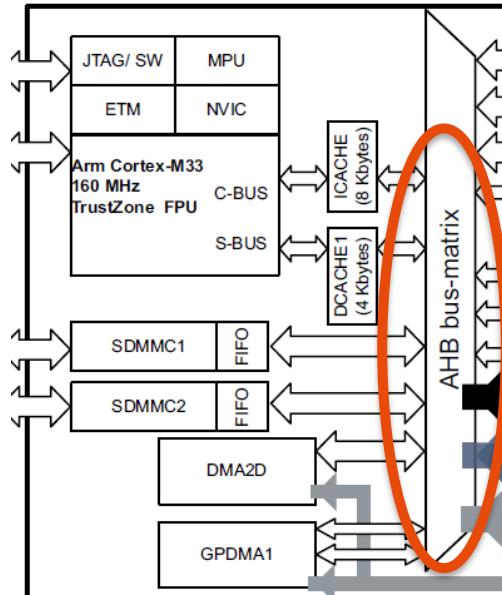
A bus is a communication system that transfers data between components of the system
STM32U5 follows the Advanced Microcontroller Bus Architecture (AMBA5). Includes:

- **Advanced High-performance Bus (AHB)**
 - Designed for high-performance, high clock frequency systems
- **Advanced Peripheral Bus (APB)**
 - Designed for low bandwidth control accesses, (i.e. register interfaces on system peripherals)
 - Address and data phase similar to AHB, but a much reduced, low complexity signal list and designed for a low frequency system with a low bit width



AHB block diagram

The AHB bus matrix

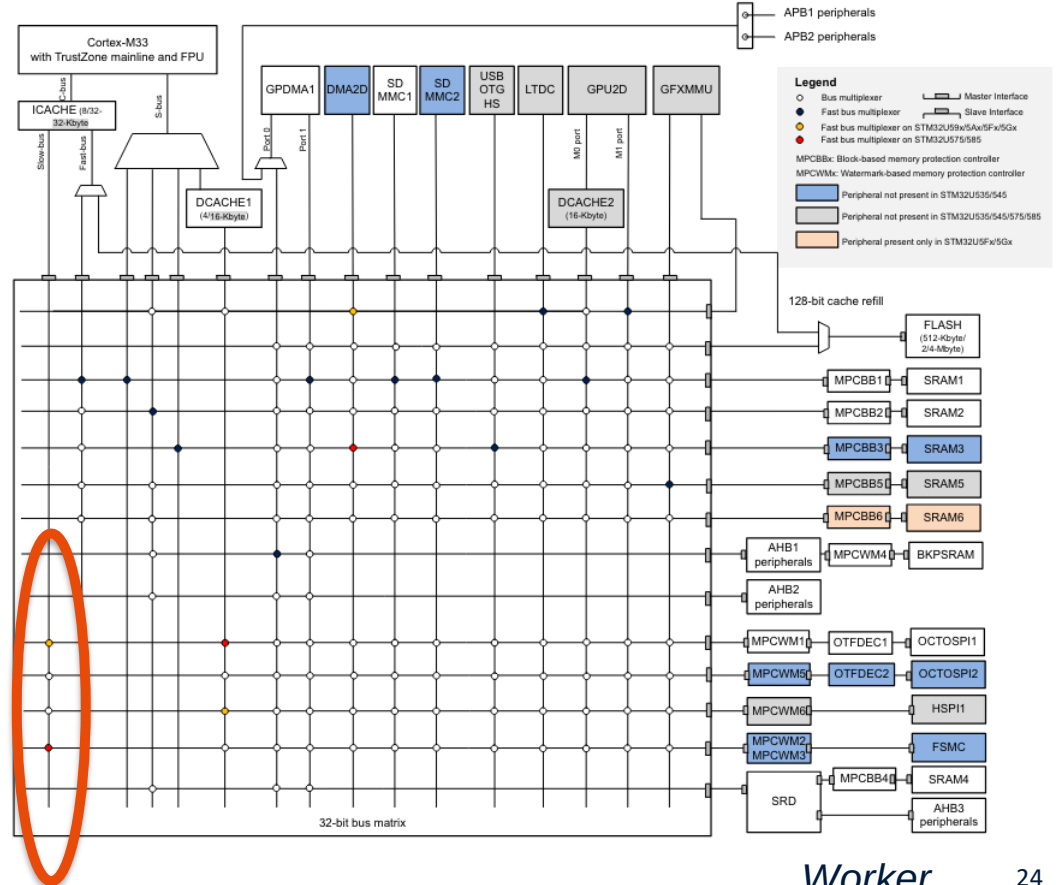


- Connects multiple masters, such as the CPU and DMA controller, to multiple workers, including memory controllers and peripheral interfaces, enabling high-speed data transfer
- Enables concurrent access and efficient operation even when several high-speed peripherals work simultaneously

The AHB bus matrix

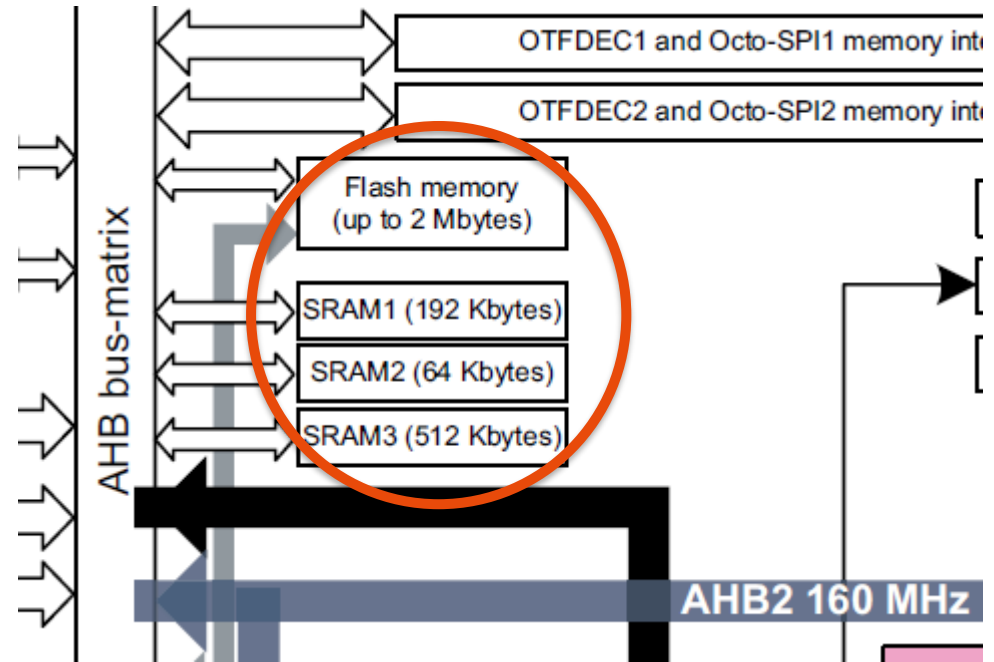
Masters

- Indicate a connection between i.e. ICACHE and SRAM or DCACHE and FLASH

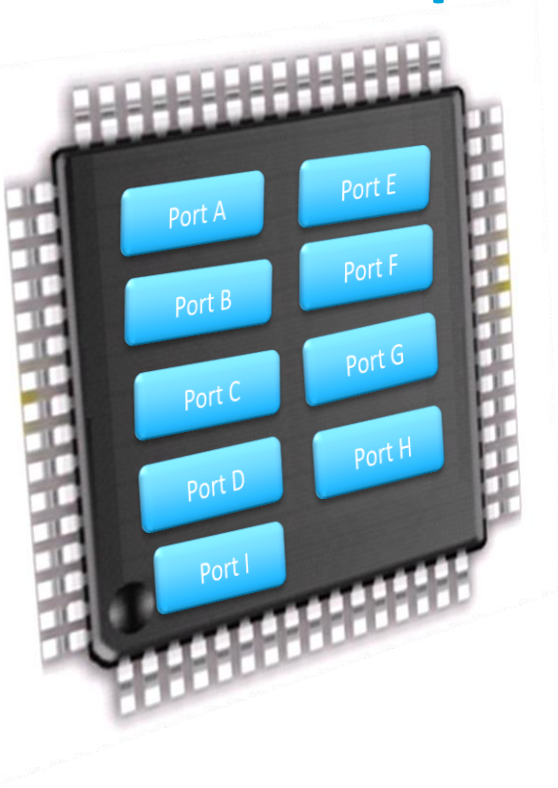


Memories

- 2Mb of embedded FLASH memory, available for storing programs and data
- Flash memory supports 10.000 cycles
- A 128-bit instruction prefetch is implemented and can optionally be enabled
- Five SRAMs are used, each with specific features. SRAM1, SRAM2, and SRAM3 are the main SRAMs

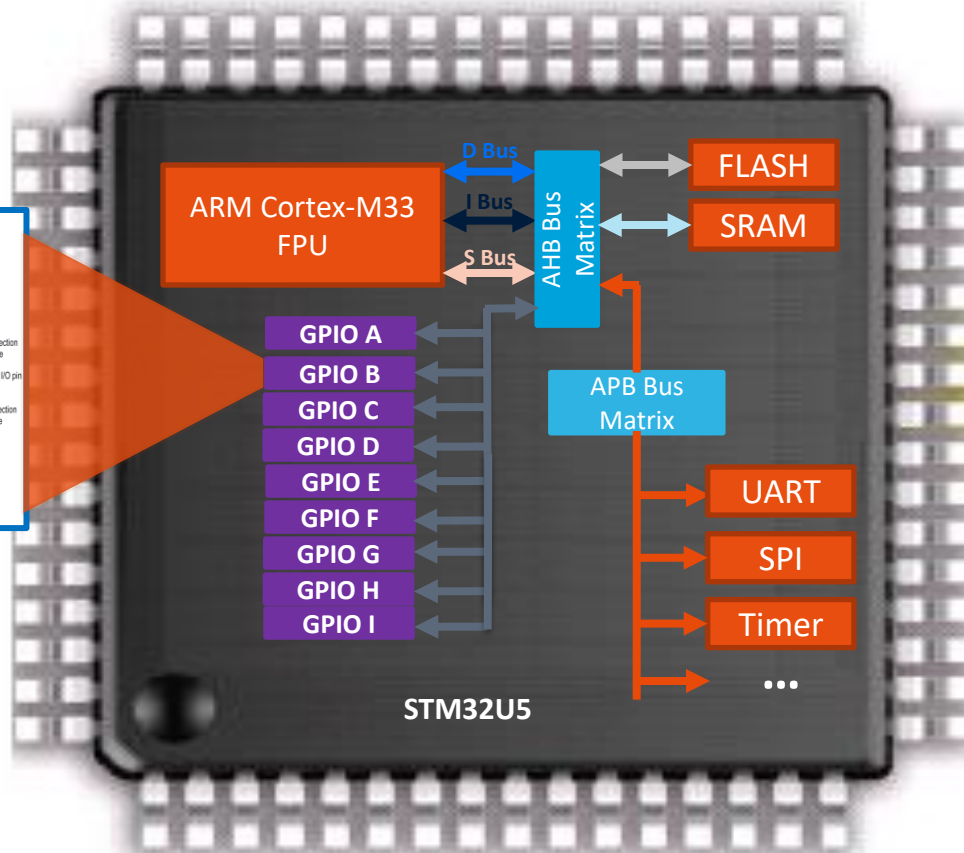
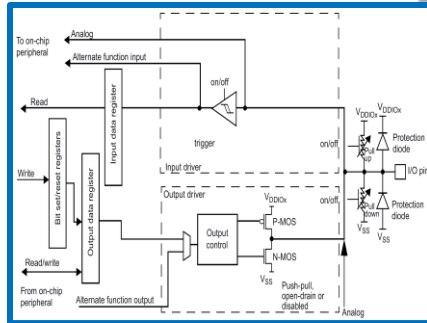


General Purpose Input/Output (GPIO)



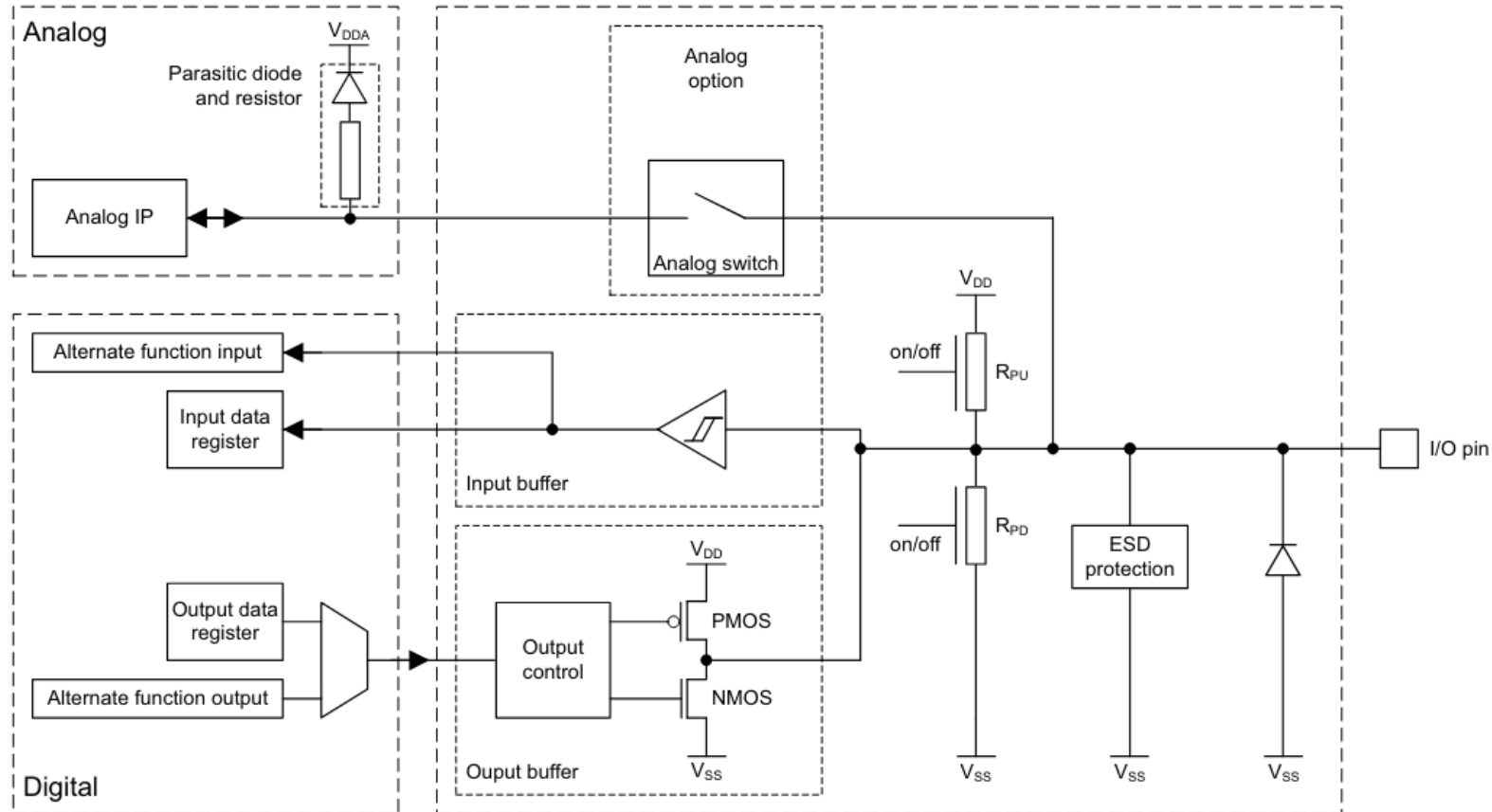
- Digital signal pin which may be used as an input or output, or both, and is controllable by software
- 9 GPIO Ports (for the 585 board, up to 10 on 59x, 5Ax, 5Fx, 5Gx): A, B, C, D, E, F, G, H, I
- Up to 16 pins in each port (Total of up to 137 GPIOs depending on MCU package)
- Each of the GPIO pins can be configured by software as output (push-pull or open-drain), as input (with or without pull-up or pull-down) or as peripheral alternate function. Most of the GPIO pins are shared with digital or analog alternate functions. After reset, all GPIOs are in analog mode to reduce power consumption.

General Purpose Input/Output (GPIO)

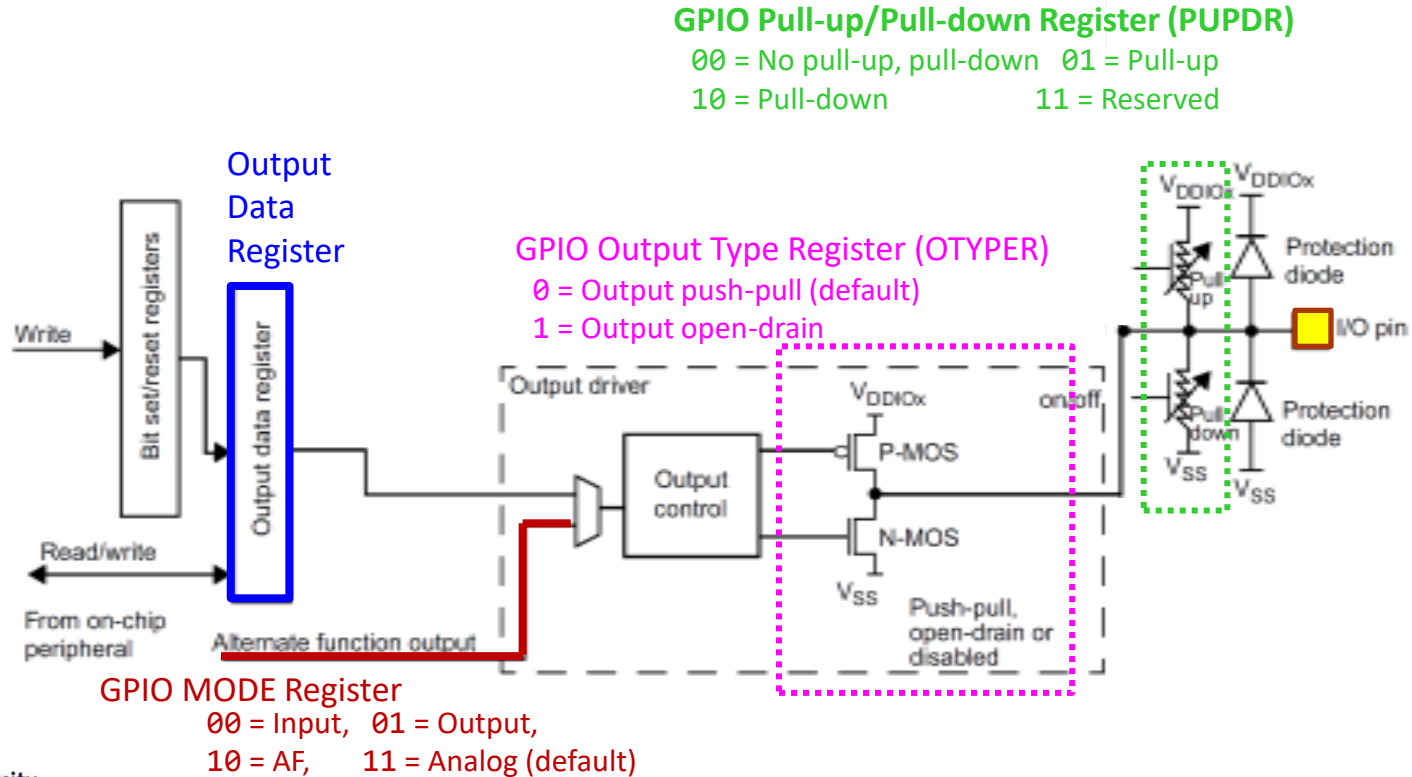


Basic Structure of an I/O Port Bit **Input** and **Output**

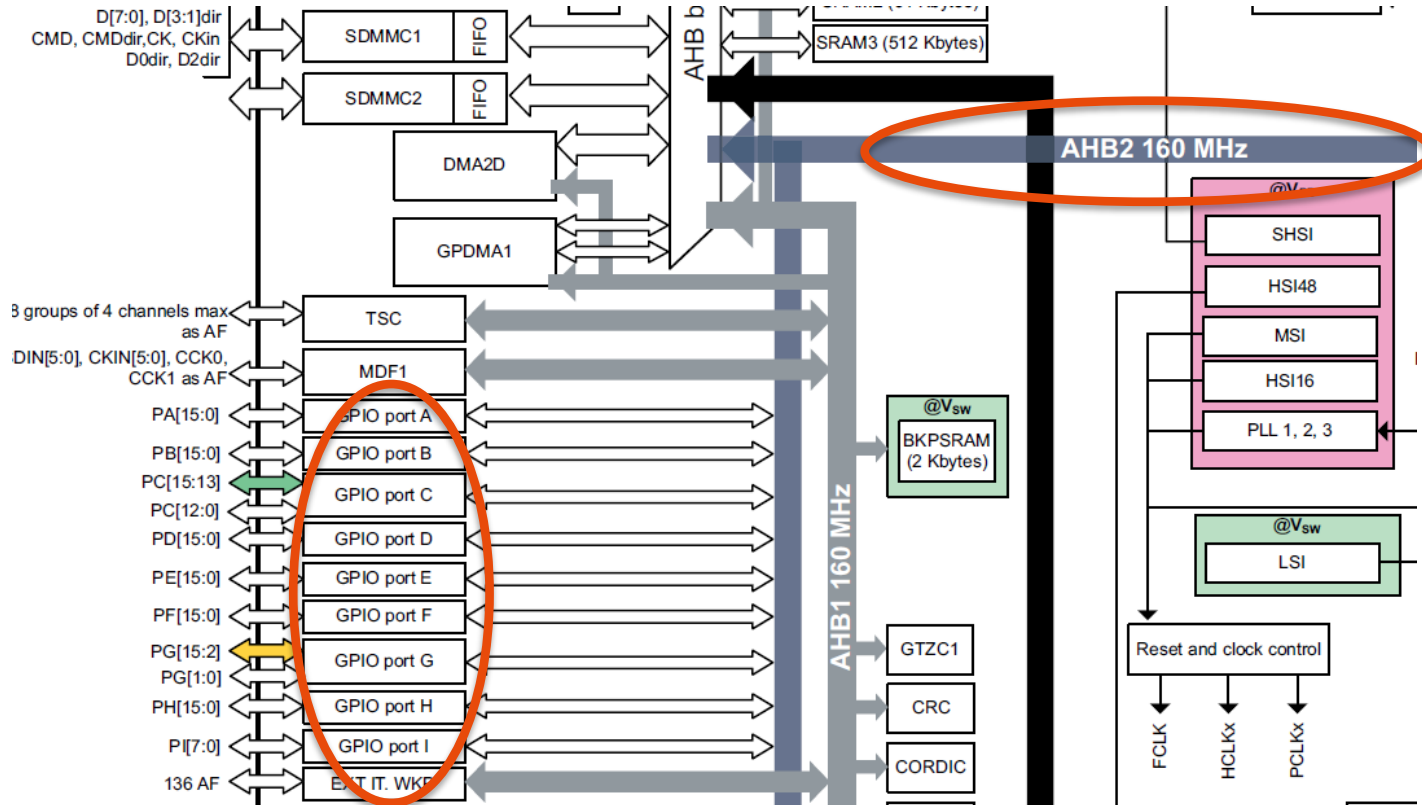
RM0456



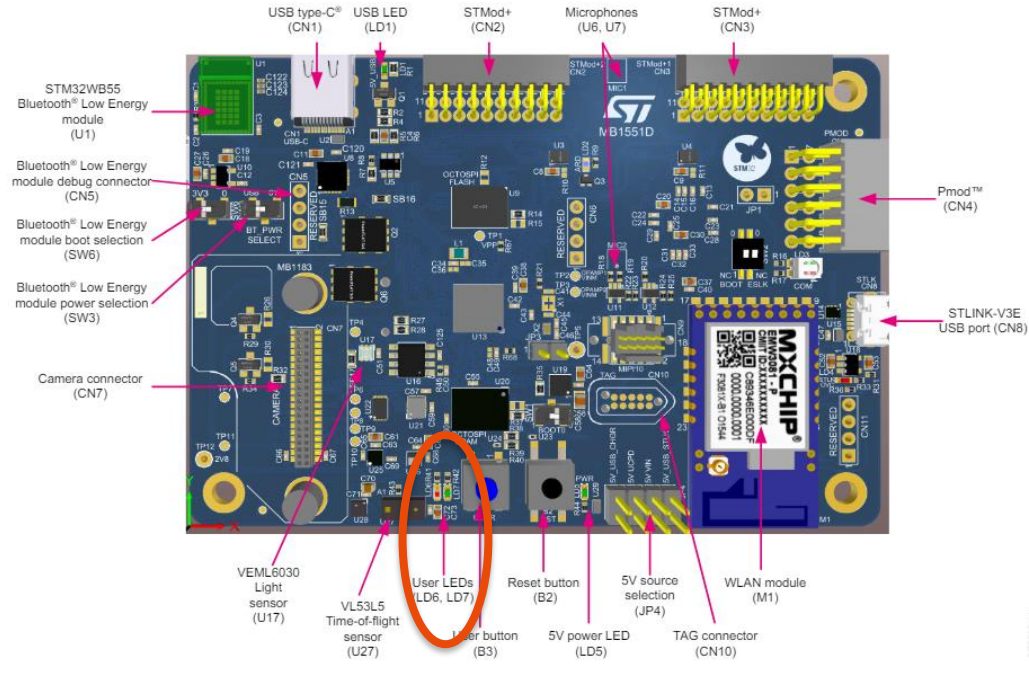
Basic Structure of an I/O Port Bit: **Output**



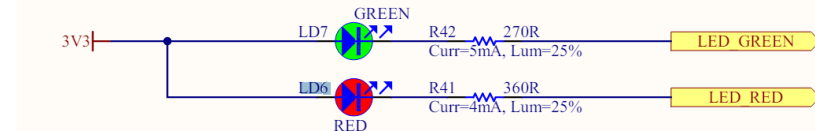
I/O Port and Peripheral Bus



The on-board LEDs of the Discovery board



USER LEDs



Enable AHB2 clock

- AHB2 peripheral clock enable register 1 (**RCC_AHB2ENR1**)

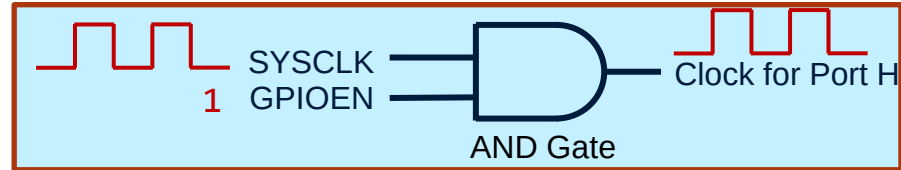
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM3 EN	SRAM2 EN	Res.	SDMM C2EN	SDMM C1EN	Res.	Res.	OTFDE C2EN	OTFDE C1EN	Res.	OCTOS PIMEN	SAESE N	PKAEN	RNGE N	HASHE N	AESEN
rw	rw		rw	rw			rw	rw		rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OTGH SPHYE N	OTGE N	Res.	DCMI PSSIE N	Res.	ADC12 EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN	GPIOD EN
rw	rw		rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 7 **GPIODEN**: I/O port H clock enable

This bit is set and cleared by software.

0: I/O port H clock disabled

1: I/O port H clock enabled



Bit 6 **GPIODEN**: I/O port G clock enable

This bit is set and cleared by software.

0: I/O port G clock disabled

1: I/O port G clock enabled

RCC->AHB2ENR1 |= RCC_AHB2ENR1_GPIODEN;

GPIO Mode Register (MODER)

- 32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Pin 7		Pin 6						Pin 2				Pin 1		Pin 0	

Bits 31:0 **MODEy[1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode

00: Input mode

01: General purpose output mode

10: Alternate function mode

11: Analog mode (reset state)

GPIO Mode Register (MODER)

- 32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Pin 7		Pin 6						Pin 2		Pin 1		Pin 0			

```
GPIOH->MODER &= ~(GPIO_MODER_MODE6_Msk); // Reset mode for red LED
```

```
GPIOH->MODER &= ~(GPIO_MODER_MODE7_Msk); // Reset mode for green LED
```

```
GPIOH->MODER |= (MY_GPIO_MODE_OUTPUT << GPIO_MODER_MODE6_Pos); // Set output mode for red LED
```

```
GPIOH->MODER |= (MY_GPIO_MODE_OUTPUT << GPIO_MODER_MODE7_Pos); // Set output mode for green LED
```

GPIO Output Type Register (OTYPE)

- 16 bits reserved, 16 data bits, 1 bit for each pin

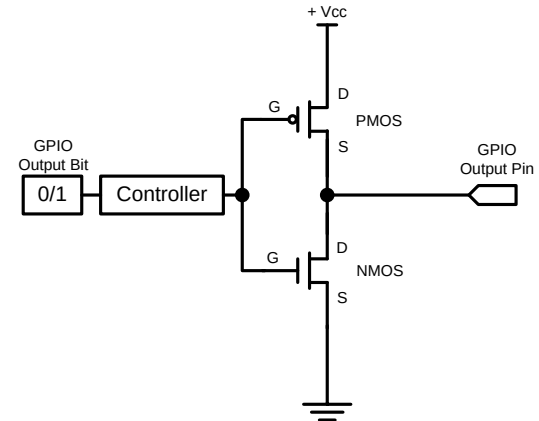
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **OTy**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O output type

0: Output push-pull (reset state)

1: Output open-drain



GPIO Output Type Register (OTYPE)

- ▶ 16 bits reserved, 16 data bits, 1 bit for each pin

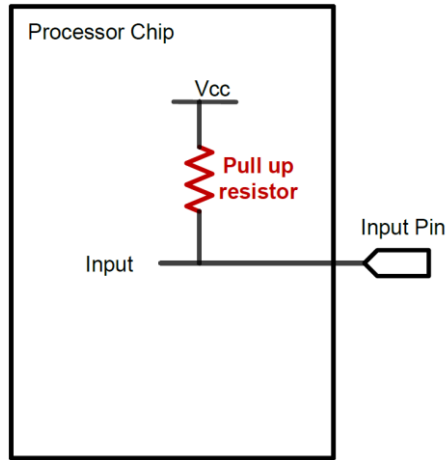
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

```
GPIOH->OTYPE &= ~(GPIO_OTYPER_OT_6); // Clear bit 6
```

```
GPIOH->OTYPE &= ~(GPIO_OTYPER_OT_7); // Clear bit 7
```

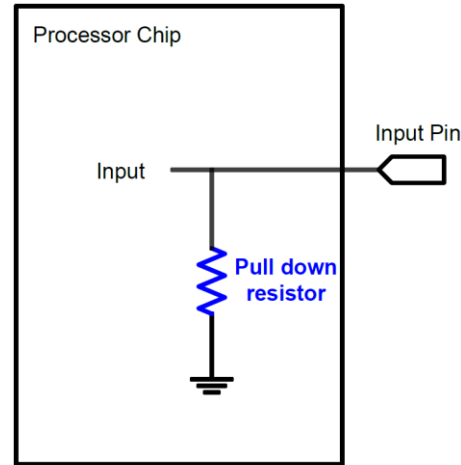
GPIO Input: Pull Up and Pull Down

- A digital input can have three states: High, Low, and High-Impedance (also called floating, tri-stated, HiZ)



Pull-Up

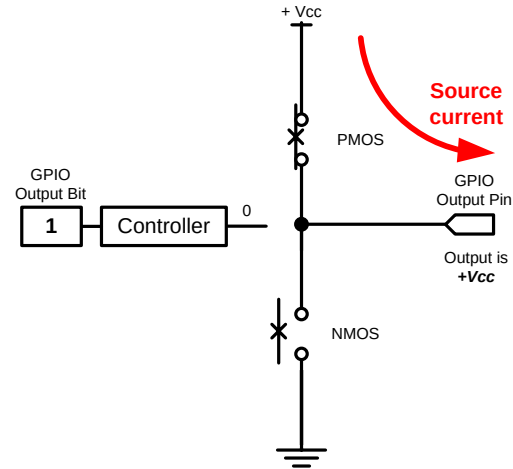
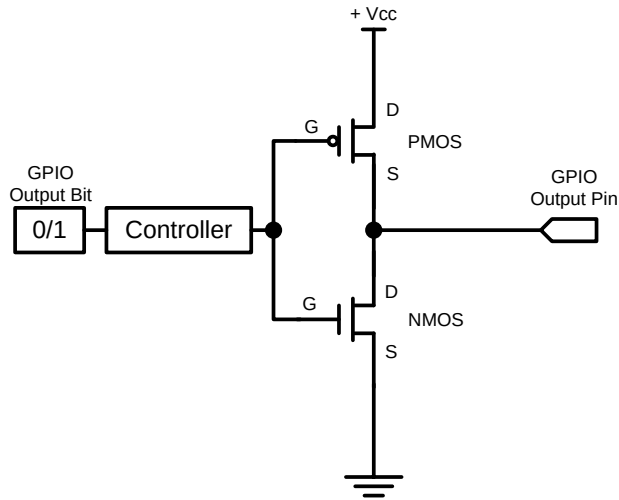
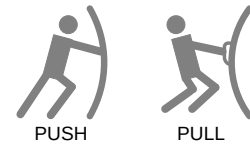
If external input is HiZ, the input is read as a valid HIGH.



Pull-Down

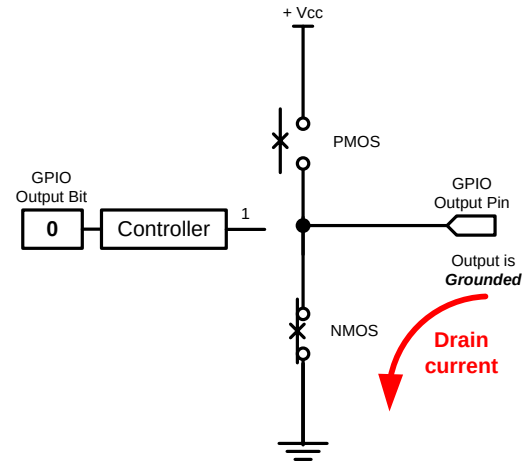
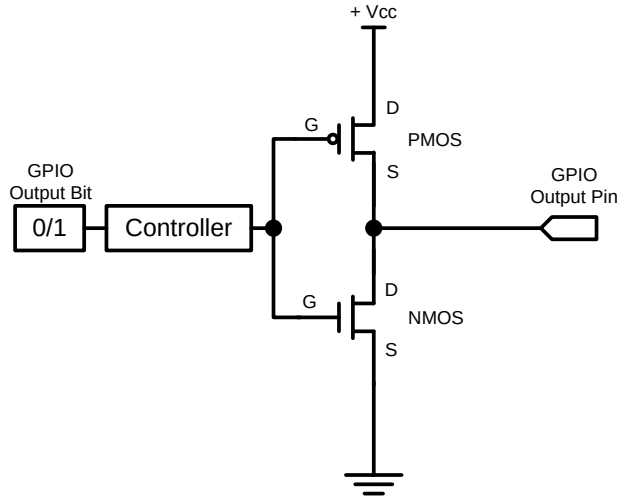
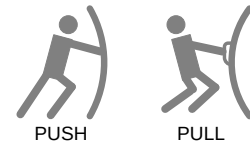
If external input is HiZ, the input is read as a valid LOW.

GPIO Output: Push-Pull



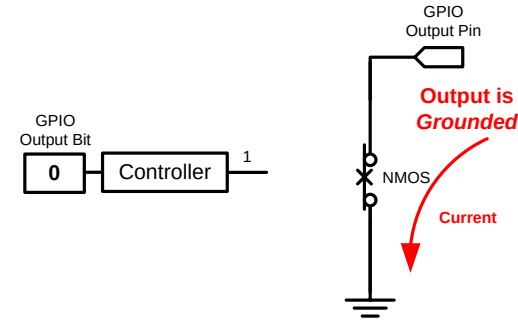
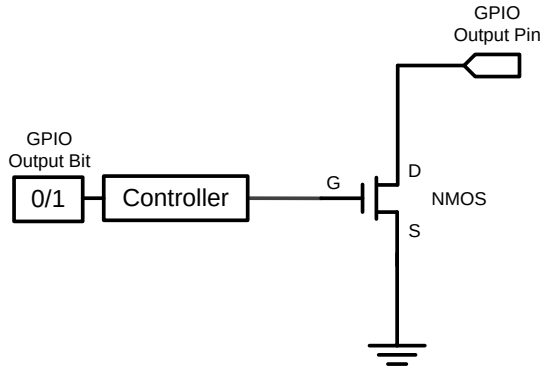
GPIO Output = 1
Source current to external circuit

GPIO Output: Push-Pull



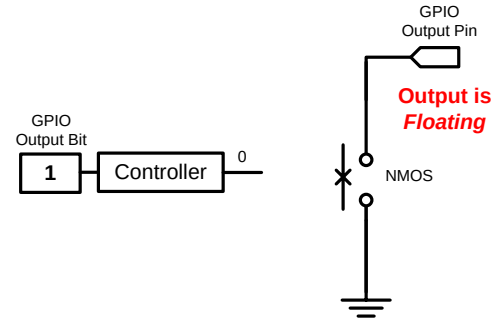
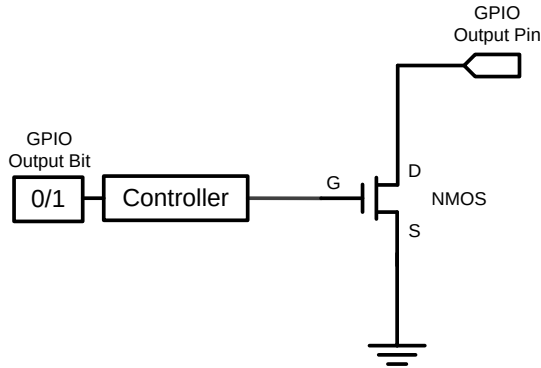
GPIO Output = 0
Drain current from external circuit

GPIO Output: Open-Drain



GPIO Output = 0
Drain current from external circuit

GPIO Output: Open-Drive



Output = 1
GPIO Pin has high-impedance to external circuit

GPIO Output: Push-Pull

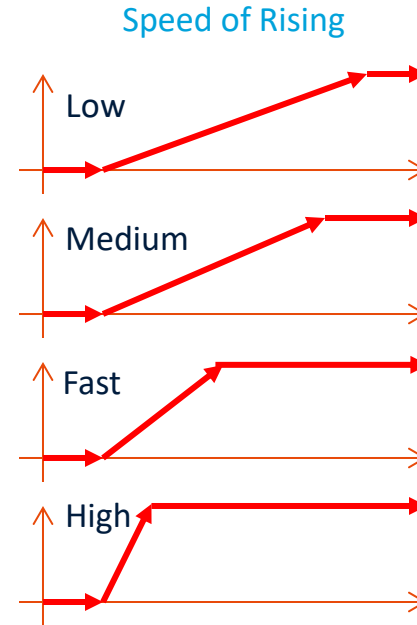
- **Configuration:** The output is actively driven both high and low using two transistors (one connected to Vcc and one to GND).
- **Operation:**
 - When output is HIGH (1), the upper transistor connects the pin to Vcc.
 - When output is LOW (0), the lower transistor connects the pin to GND
- **Advantages:**
 - Provides strong drive capability in both high and low states
 - Faster switching due to active high and low transitions
- **Disadvantages:**
 - Cannot be used in wired-AND or bus systems where multiple devices share a line
 - Direct short circuit risk if multiple devices drive different levels simultaneously
- Use Push-Pull for direct control of devices like LEDs or SPI.

GPIO Output: Open-Drain

- **Configuration:** Only one transistor (connected to GND) actively pulls the output LOW, while the high state requires an external pull-up resistor.
- **Operation:**
 - When output is LOW (0), the transistor connects the pin to GND
 - When output is HIGH (1), the transistor is turned off, and an external pull-up resistor pulls the line to Vcc
- **Advantages:**
 - Allows multiple devices to share the same line (wired-AND logic)
 - Safe for bidirectional communication (e.g., I²C)
- **Disadvantages:**
 - Requires an external pull-up resistor to drive high
 - Slower transitions due to resistor pull-up limitations
- Use Open-Drain for shared communication lines like I²C or when multiple devices need to pull a line low

GPIO Output Speed

- Output Speed:
 - Speed of rising and falling
 - Four speeds: Low, Medium, Fast, High
- Tradeoff
 - Higher GPIO speed increases EMI noise and power consumption
 - Configure based on peripheral speed
 - Low speed for toggling LEDs
 - High speed for SPI



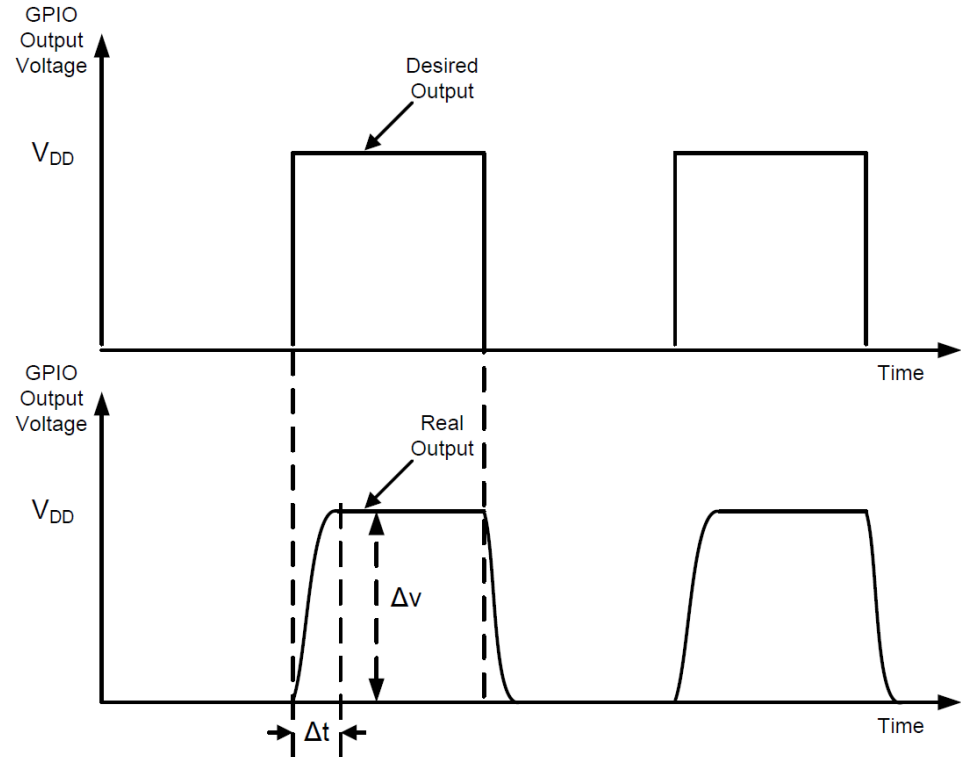
Slew Rate

Slew Rate

Maximum rate of change of the output voltage

$$\text{Slew Rate} = \max \left(\frac{\Delta V}{\Delta t} \right)$$

A high slew rate allows the output to be toggled at a fast speed.



GPIO bit set/reset register (BSRR)

- 16 high bits reset, 16 low bits set, 1 bit for each pin. Write only!

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

`GPIOH->BSRR = LED_RED_SET;`

`GPIOH->BSRR = LED_GREEN_SET;`

`GPIOH->BSRR = LED_RED_RESET;`

`GPIOH->BSRR = LED_GREEN_RESET;`

Light up the LEDs (PH6 and PH7)

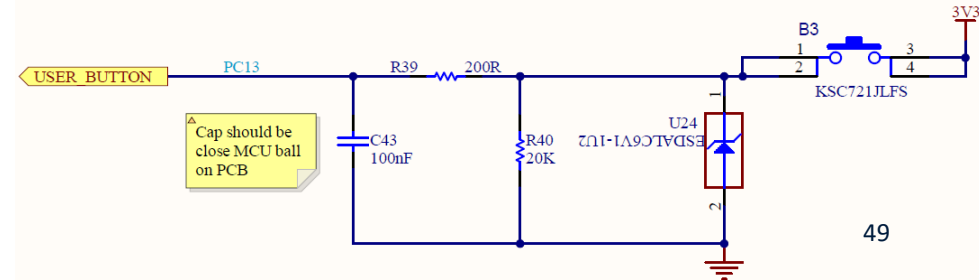
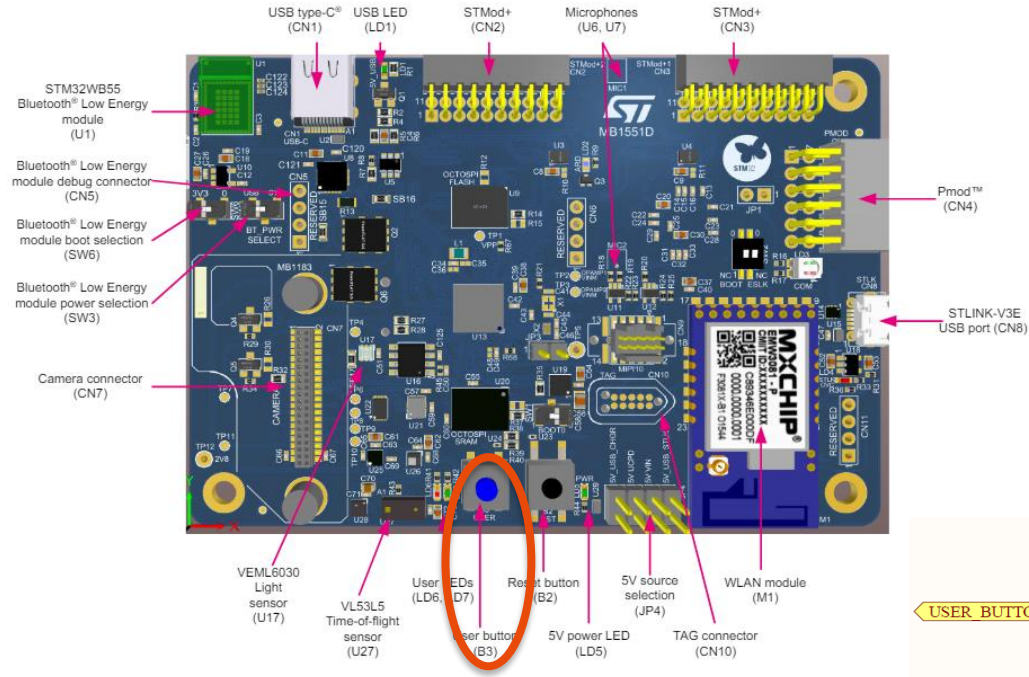
```
#define LED_RED_SET      (1U << 6)
#define LED_RED_RESET   (LED_RED_SET << 16)
#define LED_GREEN_SET   (1U << 7)
#define LED_GREEN_RESET (LED_GREEN_SET << 16)

enum {MY_GPIO_MODE_INPUT, MY_GPIO_MODE_OUTPUT, MY_GPIO_MODE_AF, MY_GPIO_MODE_ANALOG};

RCC->AHB2ENR1 |= RCC_AHB2ENR1_GPIOHEN;
GPIOH->MODER &= ~(GPIO_MODER_MODE6_Msk); // Reset mode for red LED
GPIOH->MODER &= ~(GPIO_MODER_MODE7_Msk); // Reset mode for green LED
GPIOH->MODER |= (MY_GPIO_MODE_OUTPUT << GPIO_MODER_MODE6_Pos);
GPIOH->MODER |= (MY_GPIO_MODE_OUTPUT << GPIO_MODER_MODE7_Pos);
while(1) {
    GPIOH->BSRR = LED_RED_SET;
    GPIOH->BSRR = LED_GREEN_SET;

    GPIOH->BSRR = LED_RED_RESET;
    GPIOH->BSRR = LED_GREEN_RESET;
}
```

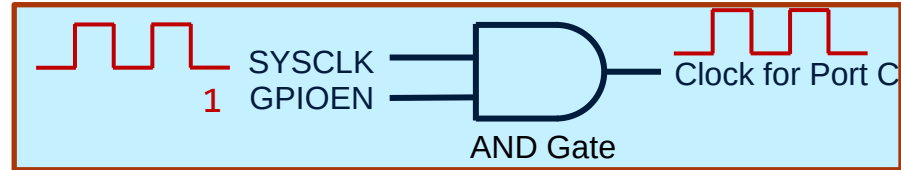
The push-button of the Discovery board



Enable AHB2 clock

- AHB2 peripheral clock enable register 1 (**RCC_AHB2ENR1**)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM3 EN	SRAM2 EN	Res.	SDMM C2EN	SDMM C1EN	Res.	Res.	OTFDE C2EN	OTFDE C1EN	Res.	OCTOS PIMEN	SAESE N	PKAEN	RNGE N	HASHE N	AESEN
rw	rw		rw	rw			rw	rw		rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OTGH SPHYE N	OTGE N	Res.	DCMI_ PSSIE N	Res.	ADC12 EN	GPIOJ EN	GPIOIE N	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
rw	rw		rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



RCC->AHB2ENR1 |= RCC_AHB2ENR1_GPIOCEN;

GPIO Mode Register (MODER)

- 32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

```
GPIOC->MODER &= ~(GPIO_MODER_MODE13_Msk);
```

```
GPIOC->MODER |= (MY_GPIO_MODE_INPUT << GPIO_MODER_MODE13_Pos);
```

GPIO Input Data Register (IDR)

- 32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

```
if ((GPIOC->IDR & GPIO_IDR_ID13) == GPIO_IDR_ID13) {  
    GPIOH->BSRR = LED_GREEN_SET;  
}
```